

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	SECTION: CPE21S1
DATE PERFORMED: August 4 2026	DATE SUBMITTED: August 4 2026
NAME: Mendoza, Roy Gabriel I.	INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. <u>5.1</u>	Progress Report Hands-on Activity 5.1 Queues
--------------------------------	---

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A:

CODE:

```
//  
// Created by royga on 8/4/2026.  
//  
#include <iostream>  
#include <queue>  
#include <string>  
  
//prototyping  
void display(std::queue<char> r);  
  
int main() {  
    //pushing  
    std::queue<char> q;  
    q.push('c');  
    q.push('p');  
    q.push('e');  
    q.push('0');  
    q.push('1');  
    q.push('0');  
  
    //display  
    std::cout<< "\n The contents of the queue are: \n";  
    display(q);  
    std::cout<< " ===== \n";  
}
```

```

//empty
std::cout<<"\n is the queue empty? " << (q.empty() ? "Yes" :
"No")<<"\n";
std::cout<< " ===== \n";

//size
std::cout<< "\n The size of the queue is " << (q.size())<<"\n";
std::cout<< " ===== \n";

//front
std::cout << "\n Front element: " << q.front() << "\n";
std::cout<< " ===== \n";

//back
std::cout << "\n Back element: " << q.back() << "\n";
std::cout<< " ===== \n";

//pop
q.pop();
std::cout << "\n After one pop:";
display(q);
std::cout<< " ===== \n";

//push
q.push('8');
std::cout << "\n After pushing '8': ";
display(q);
std::cout<< " ===== \n";

//display
std::cout << "\n Queue check: ";
display(q);
std::cout<< " ===== \n";

//deletion?
while (!q.empty()) {
    display(q);
    q.pop();
}
std::cout << "\n Final queue check: ";
display(q);
std::cout<< " ===== \n";
}

//function definitions
void display(std::queue<char> r) {
    // create a copy of the queue
    std::queue<char> c = r;

    while (!c.empty()) {
        std::cout << " " << c.front();
        c.pop();
    }
}

```

```
std::cout << "\n";  
}
```

OUTPUT:

```
"C:\Users\royga\CLionProjects\untitled1\H0A 5p1\ILOA.exe"
```

```
The contents of the queue are:
```

```
c p e 0 1 0
```

```
=====
```

```
is the queue empty? No
```

```
=====
```

```
The size of the queue is 6
```

```
=====
```

```
Front element: c
```

```
=====
```

```
Back element: 0
```

```
=====
```

```
After one pop: p e 0 1 0
```

```
=====
```

```
After pushing '8':  p e 0 1 0 8
```

```
=====
```

```
Queue check:  p e 0 1 0 8
```

```
=====
```

```
p e 0 1 0 8
```

```
e 0 1 0 8
```

```
0 1 0 8
```

```
1 0 8
```

```
0 8
```

```
8
```

```
Final queue check:
```

```
=====
```

```
Process finished with exit code 0
```

Code Analysis:

ILO A is basically part where we still use the stl of the queue, in this part we use the standard **push()**, **pop()**, **empty()**, **size()**, **front()**, **back()** and the self made **display()**. This part basically shows the major **queue** operations. Overall, this is like the stack activity but with some modifications that fits the first in first out idea.

ILO B:

CODE:

RUNNER:

```
//  
// Created by royga on 8/4/2026.  
//  
#include <iostream>  
#include <queue>  
#include "queueell.h"  
template <typename T>  
void display(qNode<T>* frontPtr);  
  
int main() {  
    qNode<char>* front = nullptr;  
    qNode<char>* back = nullptr;  
  
    enqueue(&front, &back, 'R');  
    std::cout << "Front: " << front->data << " Back: " << back->data <<  
    "\n";  
    std::cout<< " ===== \n";  
    enqueue(&front, &back, 'O');  
    std::cout << "Front: " << front->data << " Back: " << back->data <<  
    "\n";  
    std::cout<< " ===== \n";  
    enqueue(&front, &back, 'Y');  
    std::cout << "Front: " << front->data << " Back: " << back->data <<  
    "\n";  
    std::cout<< " ===== \n";  
    dequeue(&front, &back);  
    std::cout << "After dequeue: ";  
    display(front);  
    std::cout<< " ===== \n";  
    std::cout << "Front element: " << top1st(front) << "\n";  
    dequeue(&front, &back);  
    std::cout<< " ===== \n";  
    std::cout << "The new front: " << top1st(front) << "\n";  
    std::cout<< " ===== \n";  
  
    return 0;  
}
```

HEADER:

```
//  
// Created by royga on 8/4/2026.  
//  
  
#ifndef UNTITLED1_QUEUEELL_H  
#define UNTITLED1_QUEUEELL_H
```

```

template<typename T>
class qNode {
public:
    T data;
    qNode<T> *next;
};

//creating a new node:
template <typename T>
qNode<T>* new_Node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//pushing from the back
template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData) {
    //create a new node;
    qNode<T>* newNode = new_Node(newData);

    //inserting to an empty queue
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr ) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    //inserting an item into a non empty queue
    //point the backPtr next to the newNode
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    //create a temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    //check if the queue is only 1 node
    if ((*frontPtr)->next==nullptr && (*backPtr)->next==nullptr) {
        (*frontPtr)= nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }
    //deleting of the node
    (*frontPtr)= deleteNode->next;
    delete deleteNode;
}

```

```

//display all elements in the list
template <typename T>
void display(qNode<T>* frontPtr) {
    qNode<T>* current = frontPtr;
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << "\n";
}

//return the front element
template <typename T>
T toplst(qNode<T>* frontPtr) {
    if (frontPtr == nullptr) {
        throw std::runtime_error("Queue is empty");
    }
    return frontPtr->data;
}

#endif //UNTITLED1_QUEUE_H

```

OUTPUT:

```

"C:\Users\royga\CLionProjects\untitled1\H0A 5p1\ILOB.exe"
Front: R Back: R
=====
Front: R Back: 0
=====
Front: R Back: Y
=====
After dequeue: 0 Y
=====
Front element: 0
=====
After dequeue, the new front: Y
=====
Process finished with exit code 0

```

CODE ANALYSIS:

In this part of the discussion or HOA, we are using or creating our own **header or template** instead of the standard template **std::queue** already built, to use a **Linked list implementation** of the **queue**.

In the header file, we are still using the standard method of our professor to make the creation of the nodes, we then proceed to the **pushing / queuing** where we have to input the new pushed element to the back of the list.

Next is, we did the **dequeue** where we have to remove the first element by first putting it into a temporary node to store, then we have to delete it along with the node.

For output, the **display function** traverses the linked list starting from the front pointer and prints each element until the end of the list is reached.

Additionally, a **top1st function** is provided to return the front element without removing it, throwing an error if the queue is empty.

Altogether, this code or activity shows the complete linked-list queue implementation with **enqueue**, **dequeue**, **display**, and **peek** (in stack idea) operations, giving a clear view of how queues can be managed manually at the nodes.

ILO B.2:

Header Code:


```

1  > //...|
4  #ifndef UNTITLED1_NEWQARR_H
5  #define UNTITLED1_NEWQARR_H
6  #include <iostream>
7
8  //array based circular q
9  template <typename T>
10 class queueArr {
11 private:
12     T* q_array; //dynamic array?
13     size_t q_capacity; //maximum capacity of "nodes" of the array
14     size_t q_size; // no. of elements
15     int q_front; //1st element
16     int q_back; //last element
17
18     public:
19     //prototype:
20     → queueArr(size_t capacity);
21     → bool Empty();
22     → bool Full();
23
24     → size_t Size();
25     → void Clear();
26
27     → T Front();
28     → T Back();
29
30     → void Enqueue(T newData);

```

```

31 → ↵ void Dequeue();
32
33 → ↵ ~queueArr();
34
35 → ↵ queueArr(const queueArr& other); //copy constructor
36 → ↵ queueArr& operator=(const queueArr& other); //copy operator
37
38 };
39
40 //constructor:
41 template <typename T>
42 → ↵ queueArr<T>::queueArr(size_t capacity) {
43     //capacity of the array
44     q_capacity = capacity;
45
46     //size of the array
47     q_size = 0;
48
49     //initial front and back of the index
50     q_front = 0;
51     q_back = -1;
52
53     //newly allocated array
54     q_array = new T[q_capacity];
55     std::cout<< "\nConstructor class was called";
56 }
57 //size()
58 template <typename T>
59 → ↵ size_t queueArr<T>::Size() {
60     return q_size;

```

```

61     }
62
63     //empty()
64     template <typename T>
65     → bool queueArr<T>::Empty() {
66         return (q_size == 0);
67     }
68
69     //full()
70     template <typename T>
71     → bool queueArr<T>::Full() {
72         return (q_size == q_capacity);
73     }
74
75     //clear()
76     template <typename T>
77     → void queueArr<T>::Clear() {
78         q_size = 0;
79         q_front = 0;
80         q_back = -1;
81     }
82
83     //front()
84     template <typename T>
85     → T queueArr<T>::Front() {
86         //check and display if the queue is empty
87         if (Empty()) {
88             std::cout<< "\nQueue is empty";
89             return T{};
90         }

```

```

91         //return the front item
92     else {
93         return q_array[q_front];
94     }
95 }
96
97 //back()
98 template <typename T>
99 → T queueArr<T>::Back() {
100     if (Empty()) {
101 → ⚡ std::cout<< "\nQueue is empty";
102     }
103     else {
104         return q_array[q_back];
105     }
106 }
107
108 //enqueue
109 template <typename T>
110 → void queueArr<T>::Enqueue(T newData) {
111     //check if the queue is full
112     if (Full()) {
113         std::cout<< "\nQueue is full";
114         return;
115     }
116     //compute for the q_back to move it circularly
117     q_back = (q_back + 1) % q_capacity;
118     //assign the new value to the back of the queue array
119     q_array[q_back] = newData;
120     //increment the q_size

```

```

121         q_size++;
122     }
123
124     //dequeue()
125     template <typename T>
126     → void queueArr<T>::Dequeue() {
127         //check if empty
128         if (Empty()) {
129             std::cout<< "\nQueue is empty";
130             return;
131         }
132         //check for thr q_front to move it circularly
133         q_front = (q_front + 1) % q_capacity;
134
135         //decrement the queue size
136         q_size--;
137     }
138
139     //copy constructor
140     template <typename T>
141     → queueArr<T>::queueArr(const queueArr& other) {
142         //copying the capacity of the original
143         q_capacity = other.q_capacity;
144         //copy size
145         q_size = other.q_size;
146         //copy the front index
147         q_front = other.q_front;
148         //copy the back index
149         q_back = other.q_back;
150         //create a new array for the copied queue

```

```

151         q_array = new T[q_capacity];
152
153         //loop to copy every position of the original array
154         for (size_t i=0; i<q_capacity; i++) {
155             q_array[i] = other.q_array[i];
156         }
157         std::cout<<"\nCopy Constructor class was called";
158     }
159
160     // copy assignment operator
161     template <typename T>
162     → queueArr<T>& queueArr<T>::operator=(const queueArr<T>& other) {
163         if (this == &other) {
164             return *this;
165         }
166         delete[] q_array; //deletion of the old array
167
168         q_capacity = other.q_capacity;
169         q_size = other.q_size;
170         q_front = other.q_front;
171         q_back = other.q_back;
172         //create a new array
173         q_array = new T[q_capacity];
174         for (size_t i=0; i<q_capacity; i++) {
175             q_array[i] = other.q_array[i];
176         }
177         std::cout<<"\nCopy Operator class was called";
178         return *this;
179     }
180

```

```

181 //destructor
182 template <typename T>
183 queueArr<T>::~~queueArr() {
184     delete [] q_array;
185     std::cout<< "\nDestructor class was called";
186 }
187 #endif //UNTITLED1_NEWQARR_H
188

```

Runner Code:

```

1 //
2 // Created by noyga on 8/11/2026.
3 //
4 #include <iostream>
5 #include <string>
6 #include "newQArr.h"
7
8 int main() {
9     //create an object
10    queueArr<int> q_array( capacity: 10);
11
12    //enqueueing
13    std::cout << "\n===== \n";
14    q_array.Enqueue( newData: 50);
15    q_array.Enqueue( newData: 40);
16    q_array.Enqueue( newData: 10);
17    q_array.Enqueue( newData: 20);
18    q_array.Enqueue( newData: 30);
19    std::cout<<"\n===== \n";
20    std::cout<<" Elements are added.\n";
21    std::cout<<"\n===== \n";
22
23    //calling
24    std::cout<<"\n===== \n";
25    std::cout<<"QUEUE INFO";
26
27    std::cout << "\nFRONT: " << q_array.Front()<<"\nBACK: "<<q_array.Back()<<"\nSize: "<< q_array.Size()<<std::endl;
28
29    //dequeueing
30    std::cout<<"\n===== \n";

```

```

31     std::cout<<" Dequeueing";
32     q_array.Dequeue();
33     std::cout << "\nFRONT: " << q_array.Front()<<"\nBACK: " <<q_array.Back()<<"\nSize: " << q_array.Size()<<std::endl;
34
35     //copy constructor
36     std::cout<< " \n===== \n";
37     std::cout<<"Copy Constructor \n";
38     queueArr<int> copyQ( q_array);
39     std::cout<<"\n Copying Front: " <<copyQ.Front()<<std::endl;
40     std::cout<<"\n Copying Back: " <<copyQ.Back()<<std::endl;
41     std::cout<<"\n Copying Size: " <<copyQ.Size()<<std::endl;
42
43     //copy assignment
44     std::cout<< " \n===== \n";
45     std::cout<<" Copy Assignment Operator\n";
46     queueArr<int> assignQ( capacity: 10);
47     assignQ = copyQ;
48     std::cout<<"Copied Queue Front: " <<assignQ.Front()<<std::endl;
49     std::cout<<"\n Copied Queue Back: " <<assignQ.Back()<<std::endl;
50     std::cout<<"\n Copied Queue Size: " <<assignQ.Size()<<std::endl;
51
52     //clear
53     std::cout<<"\n===== \n";
54     std::cout<<"CLEARING";
55     q_array.Clear();
56     std::cout << "\nFRONT: " << q_array.Front()<<"\nBACK: " <<q_array.Back()<<"\nSize: " << q_array.Size()<<std::endl;
57     if (q_array.Empty()) {
58         std::cout<<"\n Queue is empty.";
59     }else {
60         std::cout<<"\n Queue is not empty.";

```

```

60         std::cout<<"\n Queue is not empty.";
61     }
62
63     std::cout<<"\n===== \n";
64     std::cout<<"Program End";
65
66     return 0;
67 }
68

```

Output:

"C:\Users\royga\CLionProjects\untitled1\HOA 5p1\newQArr.exe"

Constructor class was called

=====

=====

Elements are added.

=====

=====

QUEUE INFO

FRONT: 50

BACK: 30

Size: 5

=====

Dequeueing

FRONT: 40

BACK: 30

Size: 4

=====

Copy Constructor

Copy Constructor class was called

Copying Front: 40

Copying Back: 30

Copying Size: 4

=====

Copy Assignment Operator

Constructor class was called

Copy Operator class was called Copied Queue Front: 40

Copied Queue Back: 30

Copied Queue Size: 4

=====

CLEARING

FRONT:

Queue is empty0

BACK:

Queue is empty0

Size: 0

Queue is empty.

=====

Program End

Destructor class was called

Destructor class was called
Destructor class was called
Process finished with exit code 0

CODE ANALYSIS

The program of queueing using an array based circular queue demonstrates the concept of **FIFO(First-In-First-Out)**. It has also demonstrated a dynamic allocation through the use of the **pointer q_array(T* q_array)**. It also uses the **Empty() function** to check whether there are no current elements in the array, and inversely the **Full()**.

Now for the most important part, the **Enqueue() and Dequeue() functions** which adds a new element and removes the first element respectively. The **Front()**, and **Back()** functions are also part of this program, which shows the very first element and the last element of a given and created array. Then, I have also made the **Clear()** function that removes all elements of the queue, but not in the memory allocation since the **Destructor** was the one who deallocates them.

Not talking about the Copy Constructor and the Copy Operator, since I would not be that Visible in the output console, This activity overall runs and use the major queueing operations such as the **Enqueue, Dequeue, Front, Back, Size, Clear, Copying content and Memory Cleanup**, in an array based implementation of this Queue

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

INSTRUCTIONS:

4. SUPPLEMENTARY ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues

Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.

You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Output Analysis:

- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

5. Conclusions/Learnings:

Provide the following:

- Summary of lessons learned
- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

HEADER CODE:

QUEUE Header

```
5      #ifndef UNTITLED1_SUP5_1_H
6      #define UNTITLED1_SUP5_1_H
7      #include <iostream>
8      #include <string>
9
10     template <typename T>
11
12
13     class queueArr{
14     private:
15         T* q_array;
16         size_t q_size;
17         size_t q_capacity;
18         int q_front;
19         int q_back;
20
21     public:
22         //prototyping
23         //constructor
24         queueArr(size_t capacity);
25         //destructor
26         ~queueArr();
27         //copy constructor
28         queueArr(const queueArr& q);
29         //copy operator
30         queueArr& operator=(const queueArr& q);
```

```
30 //basic operations (using queueArr.h)
```

```
31
32 //basic operations
33 → bool empty();
34 → bool full();
35 → size_t size();
36 → void clear();
37 → T front();
38 → T back();
39 → void enqueue(T newData);
40 → T dequeue();
41 };
```

```
42
43 //CONSTRUCTOR
44 template <typename T>
45 → queueArr<T>::queueArr(size_t capacity) {
46     q_capacity = capacity;
47     q_size = 0;
48     q_front = 0;
49     q_back = -1;
50     q_array = new T[q_capacity];
51 }
```

```
52 //empty
53 template <typename T>
54 → bool queueArr<T>::empty(){
55     return(q_size ==0);
56 }
```

```
57 //Size
58 template <typename T>
59 → size_t queueArr<T>::size() {
60     return q_size;
61 }
```

```
62 //Full
63 template <typename T>
64 →← bool queueArr<T>::full() {
65     return (q_size == q_capacity);
66 }
67 //Clear
68 template <typename T>
69 →← void queueArr<T>::clear() {
70     q_size = 0;
71     q_front = 0;
72     q_back = -1;
73 }
74 //Front
75 template <typename T>
76 →← T queueArr<T>::front() {
77     if (empty()) {
78         std::cout<<"Queue is empty"<<std::endl;
79         return T();
80     }
81     return q_array[q_front];
82 }
83 //Back
84 template <typename T>
85 →← T queueArr<T>::back() {
86     if (empty()) {
87         std::cout<<"Queue is empty"<<std::endl;
88         return T();
89     }
90     return q_array[q_back];
91 }
```

```

92     //enqueue
93     template <typename T>
94     → void queueArr<T>::enqueue(T newData) {
95         if (full()) {
96             std::cout<<"Queue is full"<<std::endl;
97             return;
98         }
99         q_back++;
100        if (q_back == q_capacity) {
101            q_back = 0;
102        }
103        q_array[q_back] = newData;
104        q_size++;
105    }
106    //dequeue
107    template <typename T>
108    → T queueArr<T>::dequeue() {
109        if (empty()) {
110            std::cout<<"Queue is empty"<<std::endl;
111            return T();
112        }
113        T remData = q_array[q_front];
114        q_front++;
115        if (q_front == q_capacity) {
116            q_front = 0;
117        }
118        q_size--;
119        //reseting of index
120        if (q_size == 0) {
121            q_front = 0;
122            q_back = -1;
123        }
124        return remData;
125    }

```



```
160 //Destructor
161 template <typename T>
162 → ↵ queueArr<T>::~~queueArr() {
163     delete[] q_array;
164     std::cout<<"\n Destructor"<<std::endl;
165 }
166
167 #endif //UNTITLED1_SUP5_1_H
```

```

126 //Copy constructor
127 template <typename T>
128 → queueArr<T>::queueArr(const queueArr& other) {
129     q_capacity = other.q_capacity;
130     q_size = other.q_size;
131     q_front = other.q_front;
132     q_back = other.q_back;
133     q_array = new T[q_capacity];
134
135     for (size_t i = 0; i < q_size; i++) {
136         q_array[i] = other.q_array[i];
137     }
138     std::cout<<"\n Copy constructor"<<std::endl;
139 }
140 //Copy Operator
141 template <typename T>
142 → queueArr<T>& queueArr<T>::operator=(const queueArr<T>& other) {
143     if (this == &other) {
144         return *this;
145     }
146     delete[] q_array;
147
148     q_capacity = other.q_capacity;
149     q_size = other.q_size;
150     q_front = other.q_front;
151     q_back = other.q_back;
152     q_array = new T[q_capacity];
153
154     for (size_t i = 0; i < q_size; i++) {
155         q_array[i] = other.q_array[i];
156     }
157     std::cout<<"\n Copy assignment"<<std::endl;
158     return *this;
159 }

```

JOB Header

```
5  #ifndef UNTITLED1_JOB_H
6  #define UNTITLED1_JOB_H
7  > #include ...
9
10 class Job {
11 private:
12     int identi;
13     int pages;
14     std::string name;
15 public:
16     //prototyping
17     //constructor
18     Job(int id=0, std::string name="", int pages=0) {
19         identi = id;
20         this->pages = pages;
21         this->name = name;
22     }
23     //Job Info
24     void Display() const {
25         std::cout<<"Job Id Number: "<<identi<<std::endl;
26         std::cout<<"Pages: "<<pages<<std::endl;
27         std::cout<<"Name: "<<name<<std::endl;
28     }
```

```
28         }
29         //Getter
30         int GetId() const {
31             return identi;
32         }
33         int getPages() const {
34             return pages;
35         }
36         std::string getName() const {
37             return ↻ name;
38         }
39     };
40
41     #endif //UNTITLED1_JOB_H
42
```

PRINTER Header

```
5  #ifndef UNTITLED1_PRINTER_H
6  #define UNTITLED1_PRINTER_H
7  #include <string>
8  #include <iostream>
9  #include "sup5.1.h"
10 #include "Job.h"
11
12 class printer {
13     private:
14         // make a list containing all the queued print
15         queueArr<Job> jobQueue;
16     public:
17         //constructor
18         printer(int capacity = 15)
19             : jobQueue(capacity) {
20             std::cout<<"Ready to Print";
21         }
22         //Add Job
23         void AddPrint(Job newJob) {
24             if (jobQueue.full()) {
25                 std::cout<<"Print Limit already full";
26                 std::cout<< "Get A new Job:");
27                 return;
28             }
29             jobQueue.enqueue(newJob);
30             std::cout<<"Paper of: "<<newJob.GetId()<<" was added successfully";
31             std::cout<<"\nUser: "<<newJob.getName();
32             std::cout<<"\nPages: "<<newJob.getPages();
33         }
```

```
34 //single
35 void Print() {
36     if (jobQueue.empty()) {
37         std::cout<<"Print Limit already full";
38         return;
39     }
40     //Getting of the next job
41     Job current = jobQueue.dequeue();
42     std::cout<<"\n Processing Job";
43     current.Display();
44     std::cout<<"\n Job Complete";
45 }
46 //Multiple
47 void PrinAll() {
48     std::cout<<"\nPrint All Jobs";
49     while (!jobQueue.empty()) {
50         Print();
51     }
52     std::cout<<"\n All jobs complete";
53 }
```

```
54      //Queue Info
55      void ShowStats() {
56          std::cout<<"\n Queue Information:";
57          std::cout<<"\n Pendings:"<<jobQueue.size();
58
59          if (jobQueue.empty()) {
60              std::cout<<"\n Empty Job Queue";
61          } else {
62              std::cout<<"\n Next Job Queue:"<<std::endl;
63              Job nextJob = jobQueue.front();
64              nextJob.Display();
65          }
66      }
67  };
68  #endif //UNTITLED1_PRINTER_H
```

RUNNER CODE:

```
4  #include <iostream>
5  #include <string>
6  #include "sup5.1.h"
7  #include "Job.h"
8  #include "printer.h"
9
10 int main() {
11     std::cout<<" PRINTER SIMULATION.exe ";
12     std::cout<<"\n===== \n"
13
14     printer printer( capacity: 15);
15
16     //queue of jobs
17     Job job1( id: 609, name: "Roy", pages: 20);
18     Job job2( id: 420, name: "Bien", pages: 67);
19     Job job3( id: 034, name: "Carl", pages: 007);
20     Job job4( id: 678, name: "Twain", pages: 911);
21
22     printer.AddPrint( job1);
23     printer.AddPrint( job2);
24     printer.AddPrint( job3);
25     printer.AddPrint( job4);
26
27     //status info
28     printer.ShowStats();
```



```

30     //one job process
31     std::cout<<"\n Process One Job:";
32     printer.Print();
33
34     //add while have already started
35     std::cout<<"\n Queue more Job:";
36     Job job5( id: 911, name: "Stephen", pages: 80);
37     Job job6( id: 676, name: "Amiel", pages: 456);
38     Job job7( id: 906, name: "Jerry", pages: 159);
39
40     printer.AddPrint( job5);
41     printer.AddPrint( job6);
42     printer.AddPrint( job7);
43     printer.ShowStats();
44
45     //process all
46     printer.PrinAll();
47
48     //final status check
49     printer.ShowStats();
50
51     std::cout<<"\n===== ";
52     std::cout<<"\nPrinter Simulation End";
53
54     return 0;
55 }

```

OUTPUT:

"C:\Users\royga\CLionProjects\untitled1\HOA 5p1\sup5.1.exe"
 PRINTER SIMULATION.exe

```

=====
Ready to PrintPaper of: 609 was added successfully
User: Roy
Pages: 20Paper of: 420 was added successfully
User: Bien
Pages: 67Paper of: 28 was added successfully
User: Carl
Pages: 7Paper of: 678 was added successfully

```

User: Twain
Pages: 911
Queue Information:
Pendings:4
Next Job Queue:
Job Id Number: 609
Pages: 20
Name: Roy

Process One Job:
Processing JobJob Id Number: 609
Pages: 20
Name: Roy

Job Complete
Queue more Job:Paper of: 911 was added successfully
User: Stephen
Pages: 80Paper of: 676 was added successfully
User: Amiel
Pages: 456Paper of: 906 was added successfully
User: Jerry
Pages: 159
Queue Information:
Pendings:6
Next Job Queue:
Job Id Number: 420
Pages: 67
Name: Bien

Print All Jobs
Processing JobJob Id Number: 420
Pages: 67
Name: Bien

Job Complete
Processing JobJob Id Number: 28
Pages: 7
Name: Carl

Job Complete
Processing JobJob Id Number: 678
Pages: 911
Name: Twain

Job Complete
Processing JobJob Id Number: 911
Pages: 80
Name: Stephen

Job Complete
Processing JobJob Id Number: 676
Pages: 456
Name: Amiel

Job Complete
Processing JobJob Id Number: 906
Pages: 159
Name: Jerry

Job Complete
All jobs complete
Queue Information:
Pendings:0
Empty Job Queue
=====

Printer Simulation End
Destructor

Process finished with exit code 0

OUTPUT ANALYSIS:

This program of this activity shows the culmination of the processes and functions that I have learned during the topics in Queue in different kind of implementations. I have used the circular array based queue that we have used in the ILOB.2.

This new program or in this program, we First must initialize the job to be done, and by whom, by first inputting their job id, the name, and their number of pages. Unlike of a real printer where the text must be shown, here it is just page numbers and nothing more. After that, the program will output the Status or the "Queue" of the jobs to be done in order. After all of that, we have the Function for printing(or dequeuing in this case) of the first job, and we also have the printing of ALL the queued job, employee by employee, even if we add others while we are printing.

This output behaves like this because we follow the concept of the First In First Out, meaning that the queue does not allow newly added elements to pass the elements already waiting in the queue.

I believe that the array based implementation is more appropriate because we could just reuse the array positions after the element is removed. Also since this is the latest topic, this is the one that "STACKED" the last, so it should be taken out first.

CONCLUSION:

5. Conclusions/Learnings:

Provide the following:

- Summary of lessons learned
- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

Summary of the lessons Learned:

>> Overall, the culmination of the knowledge that I have gathered in this part of the coding exercises is that I have managed to learn the concept of the QUEUE, specifically the first in First out concept, unlike the Stack where the last in was the first to be out.

Linked list implementation has taught me to dynamically allocate it and use the pointer to the next node and so on. While the array based implementation is also a dynamic allocated, but with a fixed size or in this case, capacity, unlike the linked list where the memory is the limit. Overall, both uses the **front, back, empty, full, enqueue, and dequeue** with just some little tweaks to be done.

Summary of the Procedure:

>> From the procedure, It shows the use of the queue library, linked list implementation, and the array implementation. This 3 type of activity? Done in this activity first, explains the concept of queue, then we made the almost "unlimited" kind of queue, then followed by a "limited" capacity queue. Overall, procedures has explained the concept of the queue, for us to choose how we do future codes.

Analysis of the Supplementary Activity:

>> From my understanding and experience, I would say that the procedure B.2 is much harder than the supplementary. The only thing that would make the supplementary hard is the lack of steps to be followed, like what should the framework and flow be like, since from what I made, it basically used the B.2 for the job header, then the job header was refined to be more specific or to be used more in the printer header. This could all be done in the runner, but It would make the runner long and dis organize. Also my expectation of what I envision to be done is that the printer would accept atleast like a text format of data like a real printer, but with the thorough explanation of my fellow classmates, the pages is like a place holder for the content, I don't or the program does not need to read the content, but just show the amount of pages, along with the id and name of the user.

Concluding statement:

>> I dont want to point fingers, but I have already made a more complicated and rough version of B.2 since I thought that we would discuss first the array based implementation before doing it, But sadly we are tasked first before the discussion so.. What I made looked rough compared to the code made along with our professor. Overall, for some reason, the array based implementation is more easier for me rather than the linked list implementation. And as for my area of improvement, it would be the getting faster in understanding what is being sought after by the activity, since I need to think quite a long time to understand what is the expected result since I overcomplicate stuff, like a real code now, than a mockup. For example, the printer supplementary, I thought I really need something like a text file to be inputted, so I first thought, How would I

know how many words are fit in a single page, then how would I store it efficiently. It's like I am thinking of the real world counterpart instead of the, just print the amount of page, which I would first need to consult with others to know.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.